

Value Stream prediction — representation EDA

Question: for VS prediction, what text representation works best in each of the three places text is used — and can we drop summarization to save cost?

Setup (held constant across every run): - 100 tickets with ≥ 3 ground-truth VS (`--min-gt 3` `--sample 100` , fixed seed) — the hard, discriminating cases; single-VS tickets wash out differences. - `--count-mode gt` — request exactly the GT count, so micro **F1 = P = R** (one clean number per run) and no count-generosity bias. - evidence mode, the Recall selection prompt, K=6 historic neighbours, judge on.

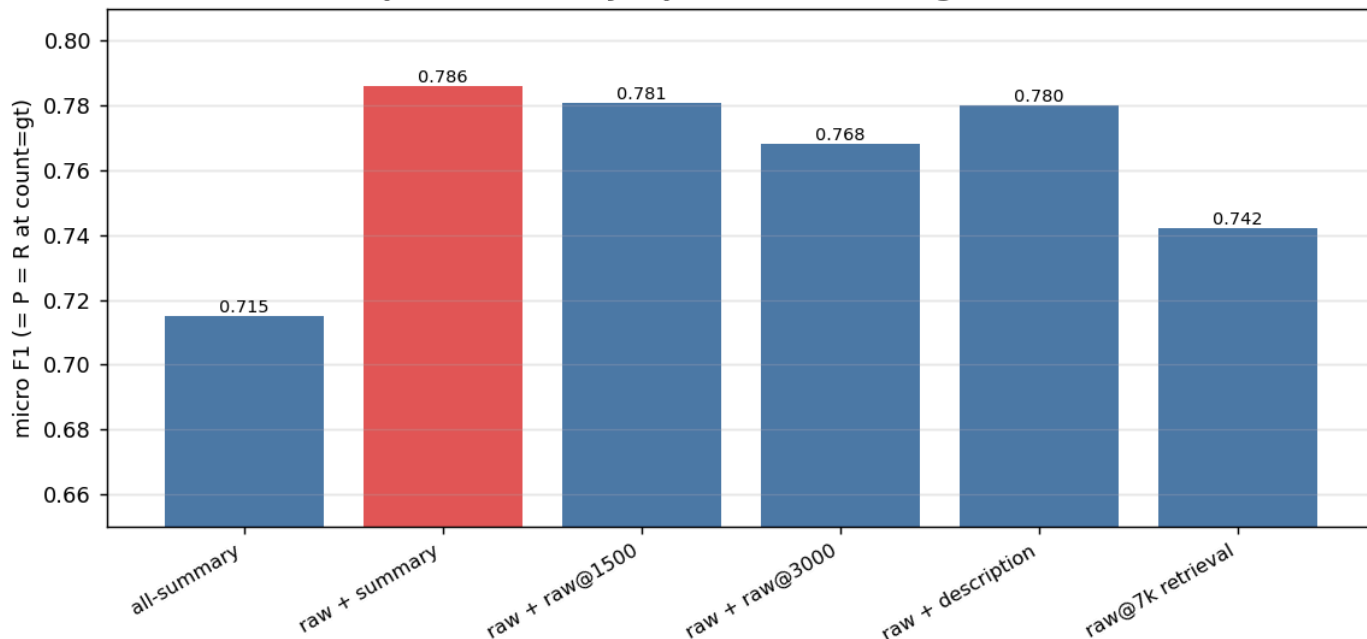
The three places text is used

place	what it does	knob
Retrieval	finds the 6 similar past tickets (embedded + searched)	summary vs raw@7k index
New-ticket prompt	the ticket context the LLM reads to pick VS	<code>--raw-text</code> (summary vs raw)
Historic block	how each of the 6 neighbours is shown in the prompt	<code>--historic-repr</code> (summary/raw/description/snippet)

These are independent. The ladder below varies one at a time.

Results

VS prediction F1 by representation (100 gt>=3 tickets)



run	prompt	historic	retrieval	F1	exact-set	avg latency
all-summary	summary	summary	summary	0.715	23%	4.2s
raw + summary	raw	summary	summary	0.786	36%	5.8s
raw + raw@1500	raw	raw@1500	summary	0.781	26%	5.6s
raw + raw@3000	raw	raw@3000	summary	0.768	24%	6.3s
raw + description	raw	description	summary	0.780	31%	4.4s
raw + raw@7k	raw	raw@7k	summary	0.780	30%	9.4s
raw@7k retrieval	raw@7k	raw@7k	raw@7k	0.742	23%	6.6s

The last row drops the summary entirely (raw-embedded index + raw everything). On a clean 100/100 with cheap raw@3k historic it lands at **0.742** — below the pack. (An earlier raw@7k-*historic* variant cost 13.9s avg / 132s max latency; dropping it to 3k fixed the latency but not the quality.)

How to read it: the big step is summary→raw on the **new-ticket prompt** (0.715 → 0.786). After that, swapping the **historic block** representation barely moves F1 (0.768–0.786) — and the last bar (raw@7k retrieval) drops *below* the pack. So the prompt is the lever, the historic block is a wash, and raw retrieval is a regression.

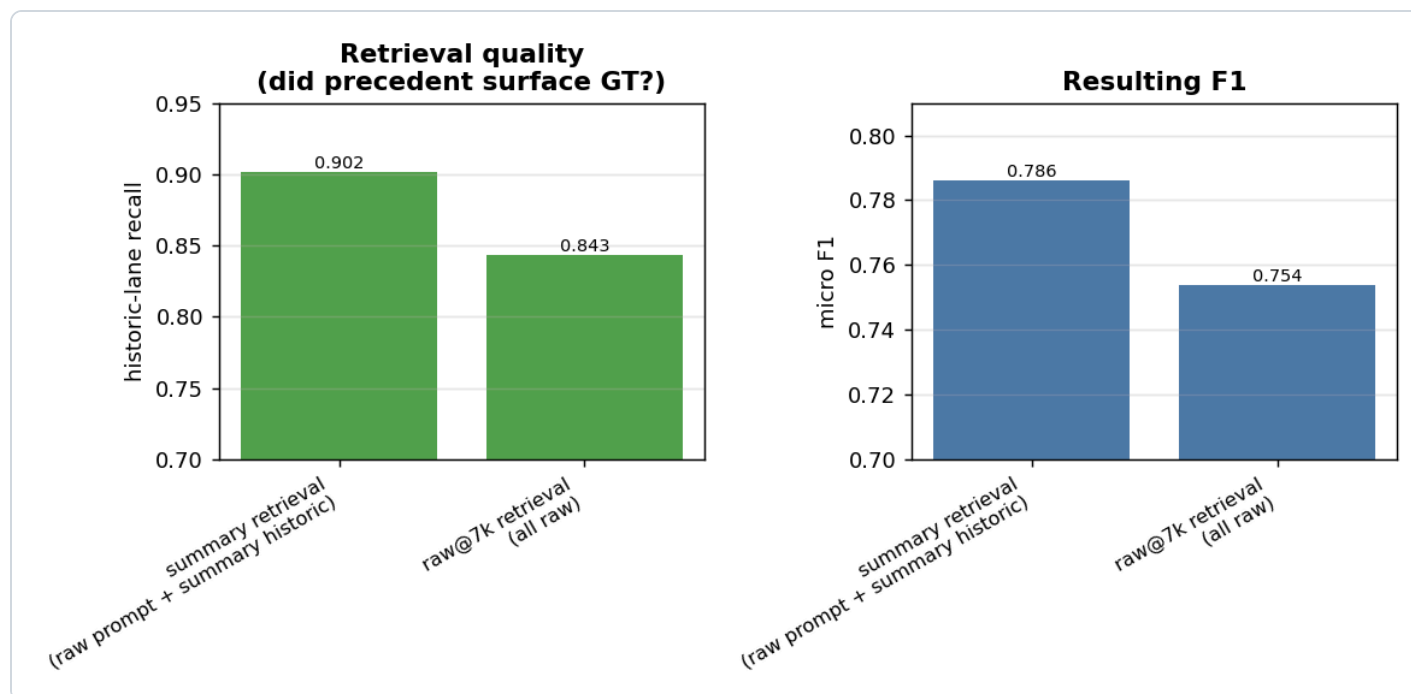
Finding 1 — the lever is the NEW-TICKET prompt

Feeding the new ticket's **raw text** instead of its summary is **+0.071 F1 (0.715 → 0.786)** and nearly doubles exact-set (23% → 36%). Retrieval is already perfect at surfacing candidates (every GT reaches the LLM), so all of this gain is the LLM *choosing* better when it sees the full ticket.

Finding 2 — the historic block representation is a wash (so use the cheapest)

Among the raw-prompt runs the historic block spans only 0.768–0.786: summary (0.786) ≈ description (0.780) ≈ raw@1500 (0.781) ≈ raw@7k (0.780), with raw@3000 slightly *worse* (0.768). More raw precedent does **not** help — it dilutes. **Summary historic is marginally best and the cheapest.**

Finding 3 — summary retrieval beats raw@7k retrieval (the cost question)

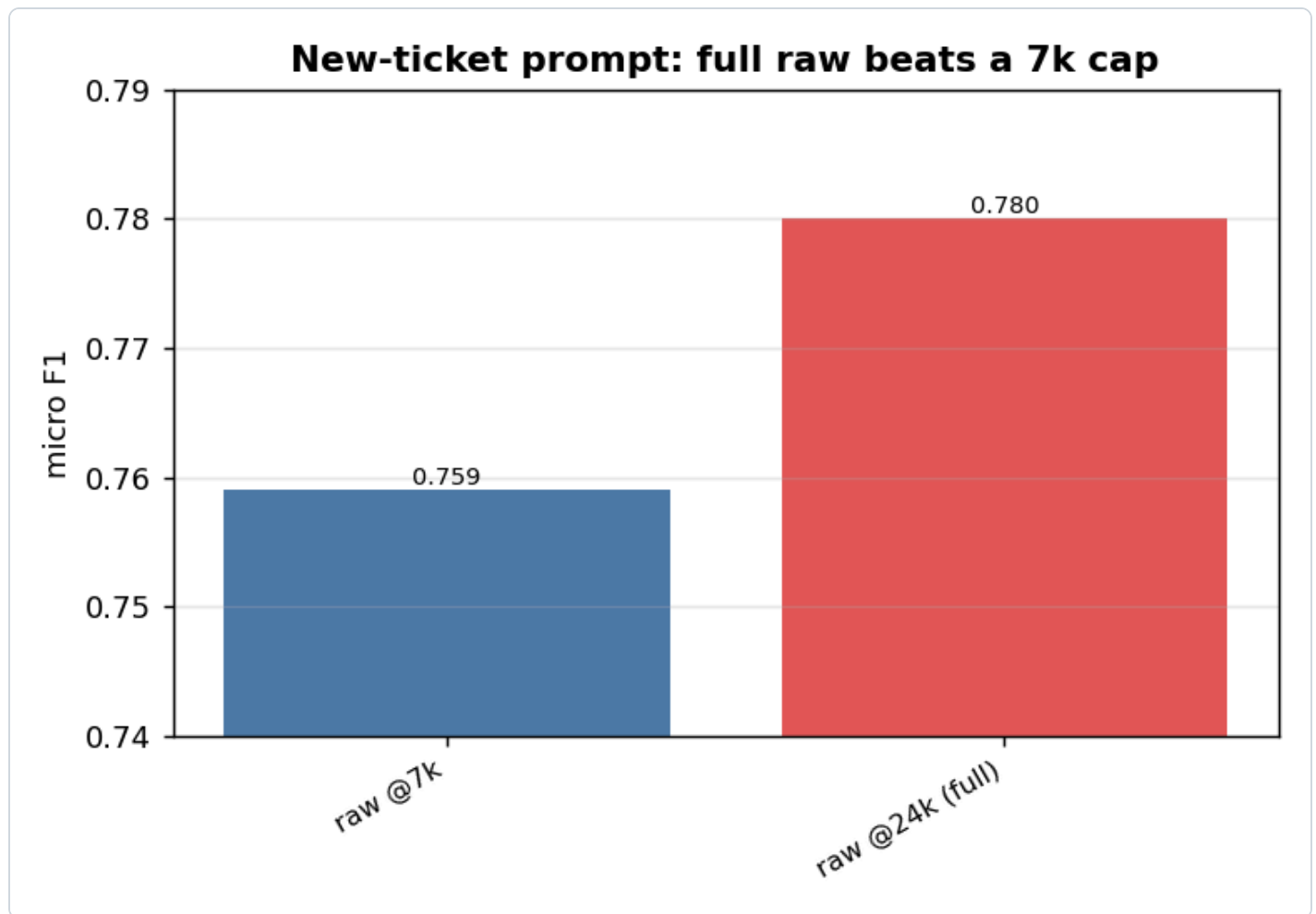


To test dropping summarization entirely, we re-embedded the index on **raw@7k** (no summary) and retrieved with raw. The decider is **historic-lane recall** — did the precedent search surface the GT (in evidence mode the 50-VS candidate pool makes review-pool recall structurally 1.0, so *that* isn't the signal — the historic lane is).

- summary retrieval: historic-lane R **0.902** → F1 **0.786**
- raw@7k retrieval: historic-lane R **0.843** → F1 **0.754**

How to read it: the ~460-token summary embeds the *whole* ticket cleanly; raw@7k keeps only ~half of a big ticket, so it retrieves **worse neighbours**. Since precedent drives recall (GT *backed* by historic is picked ~0.82, *not-backed* only ~0.38), worse precedent → lower recall → ~3 F1 points lost. **Dropping the summary costs quality.**

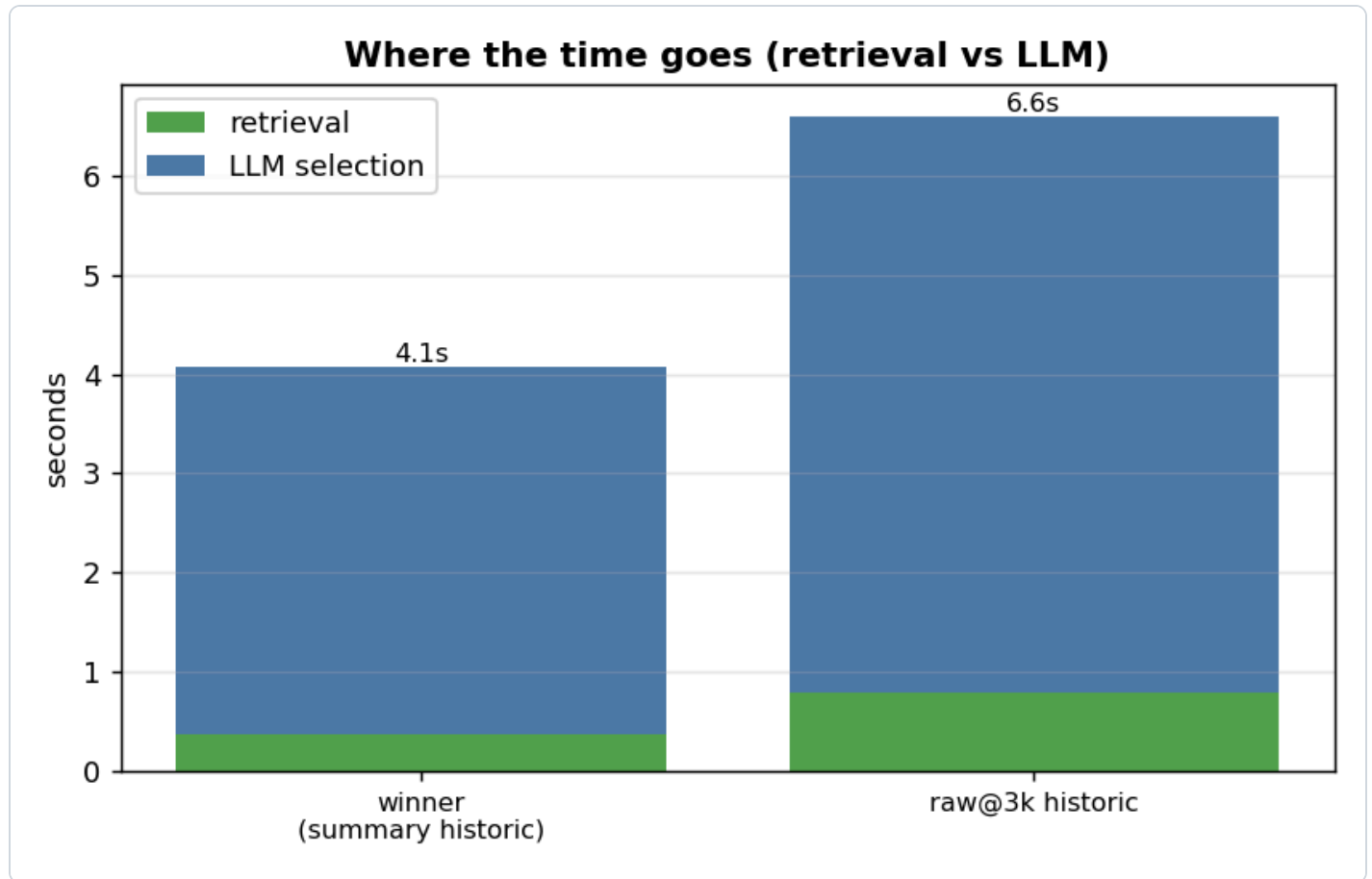
Finding 4 — feed the new ticket's FULL raw; don't cap it



Holding everything else at the winner (summary retrieval + summary historic) and changing only the new ticket's raw cap: the **full ~24k raw** scores **0.780** vs **0.759** at a 7k cap — capping loses ~2

points and drops exact-set 31% → 26%. The extra context genuinely helps the LLM decide, and latency was identical (~4s) either way, so **there's no reason to cap the new-ticket prompt**.

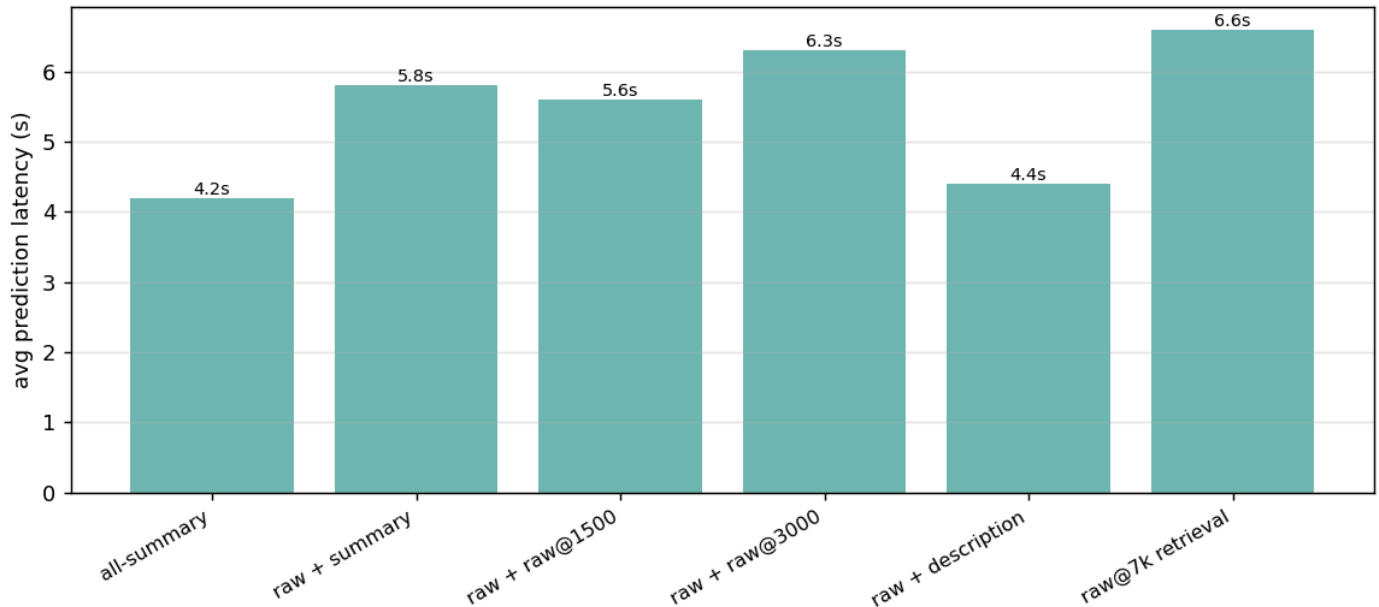
Where the time goes (latency split)



How to read it: splitting prediction latency into retrieval vs the LLM call shows **retrieval is sub-second** (0.38–0.79s) — never the bottleneck. The **LLM call is the whole cost**, and it tracks the **historic block size**: summary historic 3.7s vs raw@3k historic 5.8s. So the cheap lever for latency is the historic representation (keep it summary), not the new-ticket prompt.

Latency / cost

Latency by representation (LLM prompt size drives it)



How to read it: latency tracks the LLM **prompt size**. summary historic $\approx 6 \times 460$ tokens; raw@7k historic $\approx 6 \times 7k = 42k$ tokens, which blows the prompt to $\sim 55k$ tokens and spikes prediction to 132s on big-neighbour tickets. This is a *second* cost axis, independent of the summary question: even if you keep summaries, **never ship raw@7k historic** — it's $\sim 4\text{--}5\times$ the runtime token cost for zero quality.

Verdict

Locked config: summary retrieval + FULL $\sim 24k$ raw new-ticket prompt + summary historic — F1 $\approx 0.78\text{--}0.79$.

- The summary you "can't eliminate" turns out to be the thing buying your best retrieval.
- Dropping summarization (raw@7k retrieval) is a **regression** ($0.780 \rightarrow 0.742$, historic-lane R $0.902 \rightarrow 0.840$), not a marginal trade. The summary-free idea is dead.
- The new-ticket **raw** prompt is the real win ($+0.07$) — and feed it **in full**; a 7k cap costs ~ 2 pts.
- Use the **cheapest** historic block (summary): same quality as raw, and it's what keeps the LLM call (the only real latency) at ~ 3.7 s. Retrieval is sub-second regardless.

Final config (locked) — the per-ticket flow

1. New ticket raw, consolidated to $\leq 24k$ tokens
2. Summarize it (condense LLM) $\rightarrow$ summary fields
3. Embed the summary $\rightarrow$ search the index $\rightarrow$ retrieve the 6 nearest past tickets

4. Build the selection prompt:

- the new ticket's FULL raw (~24k) ("raw to decide")
- the 6 neighbours' summaries + their VS labels ("summaries as precedent")
- the 50-VS catalogue

5. LLM picks the VS (count = requested; evidence mode, Recall prompt)

The two uses of "summary" are different field sets, by design: - **Retrieval query** embeds **all six** summary fields (generatedSummary + businessProblem + businessCapability + keyTerms + stakeholders + systemsAndProducts). - **Historic block** shows each neighbour's **generatedSummary only** + its VS labels.

So every new ticket contributes **both** its summary (to *find* precedent) and its full raw (to *decide*) — which is exactly why summarization stays in the pipeline. **No further changes.**